Educação Profissional Paulista

Técnico em **Desenvolvimento de Sistemas**

Secretaria da Educação

SÃO PAULO
GOVERNO DO ESTADO

Introdução ao versionamento de código, Git e GitHub

Colaboração e fluxo de trabalho com o GitHub

Aula 1: Compartilhamento e permissões

Código da aula: SISANO2C5B1S4A1

Secretaria da Educação
SÃO PAULO GOVERNO DO ESTADO

Mapa da Unidade 1 Componente 5

Introdução ao versionamento de código, Git e GitHub

semana 1

Fundamentos do versionamento de código

semana 2

Conhecendo as ferramentas existentes no mercado

semana 3

Introdução ao GitHub

Você está aqui!

semana 4

Colaboração e fluxo de trabalho com o GitHub

Introdução ao versionamento de código, Git e GitHub

Mapa da Unidade 1 Componente 5

Você está aqui!

Colaboração e fluxo de trabalho com o GitHub

4

Aula 1: Compartilhamento e permissões

Código da aula: SISANO2C5B1S4A1

Objetivos da aula

- Compartilhar repositórios e definir permissões de acesso.

Recursos didáticos

- Roteiro de Atividade Prática.
- Recurso audiovisual para exibição de vídeos e imagens.
- Caneta e caderno para anotações.
- Computador com internet.

Duração da aula

50 minutos.

Habilidades técnicas

- Compartilhar repositórios, colaborar com outros usuários e acompanhar alterações em equipe.

Habilidades socioemocionais

- Valorizar a colaboração e a troca de experiências, demonstrando interesse em aprender a dividir tarefas com responsabilidade e segurança.

Colocando em **prática**

Gerenciando uma equipe no GitHub – Simulação real

Materiais necessários

- Computador com ambiente de desenvolvimento;
- Editor de texto ou IDE para pseudocódigo;
- Caderno para rascunhos;
- Caneta.

Passo a passo

- Leiam com atenção a atividade proposta no roteiro.
- Antes de iniciar a codificação, façam esboços no papel para definir a lógica.
- Utilizem as ferramentas de desenvolvimento disponíveis para implementar a atividade.

45 minutos

Em grupos

Baixe o roteiro dessa atividade

Pause e **responda**

Registro

Em um projeto crítico, qual configuração de permissão é mais segura para uma pessoa desenvolvedora que está começando na carreira, ainda não possui conhecimento sobre o projeto e suas respectivas regras de negócio e deseja apenas conhecê-lo para que possa começar sua atuação?

- Admin – para aprender tudo
- Write – para contribuir livremente
- Read – apenas visualização
- Triage – gerenciar issues e PRs

Pause e **responda**

Registro

Em um projeto crítico, qual configuração de permissão é mais segura para uma pessoa desenvolvedora que está começando na carreira, ainda não possui conhecimento sobre o projeto e suas respectivas regras de negócio e deseja apenas conhecê-lo para que possa começar sua atuação?

- ✗ Admin – para aprender tudo
- ✗ Write – para contribuir livremente
- ✓ Read – apenas visualização
- ✗ Triage – gerenciar issues e PRs

O que nós **aprendemos hoje?**

Então ficamos assim...

1. Compreendemos como organizar equipes usando Organizations para projetos menores.

2. Entendemos como colaborar de forma segura e organizada.

3. Descobrimos como configurar três níveis principais de permissão: Read, Write e Admin.

Saiba mais

Quer aprender Git do zero em menos de uma aula?

No vídeo a seguir, você vai ver uma demonstração visual passo a passo, em português, com exemplos claros e didática envolvente que facilitam a vida de quem está começando.

CÓDIGO FONTE TV. Git: minicurso para você sair do zero! (Aprenda em 45 minutos). Disponível em: https://www.youtube.com/watch?v=ts-H3W1uLMM. Acesso em: 8 set. 2025.

Referências da aula

ALURA. **O que é Git e GitHub: os primeiros passos nessas ferramentas**, 16 nov. 2020. Disponível em: https://www.alura.com.br/artigos/o-que-e-git-github. Acesso em: 8 set. 2025.

CÓDIGO FONTE TV. Git: minicurso para você sair do zero! (Aprenda em 45 minutos). Disponível em: https://www.youtube.com/watch?v=ts-H3W1uLMM. Acesso em: 8 set. 2025.

FERNANDES, D. Git & GitHub: o que é? Por quê? Como iniciar? **Rocketseat**, [s.d.]. Disponível em: https://www.rocketseat.com.br/blog/artigos/post/git-e-github-o-que-e-por-que-como-iniciar. Acesso em: 8 set. 2025.

Identidade visual: imagens © Getty Images

Orientações ao professor

Slide 6

Orientações: compreender na prática como funcionam permissões no GitHub, regras de proteção de branches, uso de Pull Requests com revisão e o princípio do menor privilégio.

Tempo previsto: 44 minutos.

Gestão de aula: forme grupos de **3 a 5 pessoas** e defina os papéis por grupo:
Admin (gerente do projeto);
Developer (desenvolvedor);
Reviewer/QA; e
observador para registrar evidências (prints, anotações) (opcional).

Condução da dinâmica:

1. Alinhe os alunos com a situação fictícia e o objetivo ("organizar profissionalmente um repositório").
2. Exercício 1: Criar o repositório e habilitar branch protection.
3. Exercício 2: Convidar colaboradores e validar permissões (Write, Read).
   - Pessoa B cria arquivo.
   - Pessoa C tenta editar (deve falhar), mas consegue abrir issue e comentar.
4. Exercício 3: Pessoa B cria branch e PR; Pessoa A revisa, aprova e faz merge; depois, os papéis se alteram com segurança:
   - Pessoa C → Write;
   - Pessoa B → Admin (apenas para discussão, não como prática padrão).
5. Entrega do grupo:
   - Prints de branch rules, convites, PR aprovado e checks.
   - Arquivo CODEOWNERS simples (ex.: * @adminUser @reviewerUser).
   - Respostas das 3 perguntas dissertativas.
6. Após a atividade principal, há um bloco opcional com propostas de ampliação (para grupos avançados) e âncoras de aprendizado que podem ser reforçadas no fechamento da aula.

Slide 6

Expectativas de resposta:

1) Por que não conceder acesso administrativo (nível máximo) para todos, uma vez que isso facilitaria o trabalho?

Menor privilégio (least privilege): diminui blast radius de erros acidentais (ex.: apagar repo, alterar regras de proteção) e de incidentes de segurança.

Governança e auditoria: papéis distintos permitem trilhas de auditoria claras (quem configurou o quê).

Qualidade do processo: com menos Admin, todos passam por PR + revisão + checks — menos riscos de "atalhos".

Conformidade: boas práticas de compliance (LGPD/ISO) pedem segregação de funções.

2) Em que situação você daria permissão Write (escrita) para uma pessoa que está começando a desenvolver na empresa?

Quando a pessoa já fez onboarding (2FA ativado, padrões de commit/mensagens, uso de PR), precisa criar branches no repositório para usar a CI do projeto e trabalhar sob branch protection (sem push direto na main).

Contextos seguros: tarefas de baixa criticidade, feature flags, projetos internos/educacionais.

Condições: PR obrigatório com 1+ aprovação, checks passando, CODEOWNERS, commits assinados (opcional).

Se ainda estiver nos primeiros dias, pode começar em Triage + fork; depois evoluir para Write.

3) Como as branch protection rules (regras de proteção para as branches) ajudam mesmo com pessoas tendo permissão de escrita?

Obrigam PR e revisão antes de merge (ninguém altera main sozinho).

Status checks garantem que testes/CI passaram.

Conversas resolvidas e histórico linear mantêm o repositório saudável.

Restrict who can push bloqueia push direto e força o fluxo via PR.

Include administrators evita exceções: até Admin joga o mesmo jogo.

Continua...

Slide 6

Bloco opcional:

Ampliação/desafio (grupos avançados)

Adicionar GitHub Actions simples (rodar testes/linter) como status check obrigatório.

PULL_REQUEST_TEMPLATE com checklist (teste, documentação, issue vinculada).

Habilitar squash merge e discutir histórico linear.

Âncoras de aprendizado esperadas

Main protegida; ninguém “fura” o processo.

PRs com revisão e checks criam qualidade repetível.

Papéis diferentes → governança melhor e menos risco.

CODEOWNERS + status checks + squash = fluxo previsível.

Slides 7 e 8

Orientações: professor(a), use para verificar a compreensão básica do conceito. Explique por que cada alternativa está correta ou incorreta. Conecte com exemplos do cotidiano.

Tempo previsto: 6 minutos.

Alternativa correta: Read – apenas visualização.

Feedback: Muito bom! A configuração “Read only” permite que o desenvolvedor júnior aprenda observando sem risco de danos. Ele pode clonar o projeto e praticar localmente, com uma progressão gradual de permissões à medida que avança.

Educação Profissional Paulista

Técnico em **Desenvolvimento de Sistemas**

Secretaria da Educação

SÃO PAULO
GOVERNO DO ESTADO